Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б. Н. Ельцина»
ИРИТ-РТФ

ЛАБОРАТОРНАЯ РАБОТА №6

Суффиксные деревья и поиск наибольшего палиндрома

ОТЧЕТ

Студент: Киприн Сергей Андреевич РИМ-250950

Екатеринбург 2026

Задачи

1. На основе полученных в прошлой лабораторной знаний о Trie, реализовать суффиксное дерево.

2. На основе суффиксного дерева решить задачу о поиске палиндрома наибольшей длины в заданной строке.

Ссылка на репозиторий Github

https://github.com/angst-storm/js-2026

Ход выполнения

Суффиксное дерево реализовано как обычный бор: в него по очереди добавляются все суффиксы строки с маркером конца $. Общие начала суффиксов не дублируются, а сходятся в одних и тех же узлах. Поэтому любая подстрока — это путь от корня, и поиск сводится к проходу по буквам образца.

Сложность построения: $O(n^2)$, поиск подстроки: $O(m)$.

```
class SuffixNode {
  constructor() {
    this.children = new Map();
    this.starts = [];
  }
}

export class SuffixTree {
  constructor(text) {
    this.text = text;
    this.root = new SuffixNode();
    this.nodeCount = 1;
    this._build(text);
  }

  _build(text) {
    const marked = text + '$';

    for (let i = 0; i < marked.length; i++) {
      let node = this.root;

      for (let j = i; j < marked.length; j++) {
```

```
            const char = marked[j];

            if (!node.children.has(char)) {
              node.children.set(char, new SuffixNode());
              this.nodeCount += 1;
            }

            node = node.children.get(char);
          }

          node.starts.push(i);
        }
      }

      contains(pattern) {
        if (pattern.length === 0) return true;

        let node = this.root;

        for (const char of pattern) {
          if (char === '$' || !node.children.has(char)) {
            return false;
          }
          node = node.children.get(char);
        }

        return true;
      }

      findAll(pattern) {
        if (pattern.length === 0) {
          return Array.from({ length: this.text.length }, (_, i) => i);
        }

        let node = this.root;

        for (const char of pattern) {
          if (char === '$' || !node.children.has(char)) {
            return [];
          }
          node = node.children.get(char);
        }

        const positions = [];
        this._collectStarts(node, positions);
        return positions.sort((a, b) => a - b);
      }

      _collectStarts(node, out) {
        for (const start of node.starts) {
          if (start < this.text.length) out.push(start);
        }
        for (const child of node.children.values()) {
          this._collectStarts(child, out);
        }
      }
}
```

Палиндром читается одинаково слева направо и справа налево. Для нахождения палиндрома набольшей длины берём строку S, строим по ней суффиксное дерево и строим реверс строки reverse(S). Идём по суффиксам реверса вниз по дереву: пока буквы совпадают, у нас есть общая подстрока S и reverse(S). Палиндромами являются только те строки, что стоят в зеркальных местах исходной строки: начало i = n - j - len. Если в этой позиции подстрока S совпал с подстрокой реверса — это палиндром. Среди всех таких вариантов выбираем самый длинный.

```
    export function longestPalindrome(text) {
      const n = text.length;
      if (n <= 1) return text;

      const rev = [...text].reverse().join('');
      const tree = new SuffixTree(text);

      let bestStart = 0;
      let bestLen = 1;

      for (let j = 0; j < n; j++) {
        let node = tree.root;

        for (let k = j; k < n; k++) {
          const char = rev[k];
          if (!node.children.has(char)) break;

          node = node.children.get(char);
          const len = k - j + 1;
          if (len <= bestLen) continue;

          // rev[j..j+len) зеркалит text[i..i+len) при i = n - j
- len
          const i = n - j - len;
          if (i < 0) continue;

          if (text.slice(i, i + len) === rev.slice(j, j + len))
{
            bestLen = len;
            bestStart = i;
          }
        }
      }

      return text.slice(bestStart, bestStart + bestLen);
    }
```

Вывод

В ходе работы реализовано суффиксное дерево и с его помощью решена задача поиска палиндрома наибольшей длины.